

Lightweight Occlusion Removal through PEPSI++

writer: Federico Donati 2133749

course: Computer Vision

I. OUTLINE

This presentation aims to address various possible solutions to the image inpainting problem, first the problem statement will be addressed and will be described the state of the art on how current methods approach such a problem.

Then two new methods will be explored on their behavior with inpainting, both methods take inspiration from previous work, one of them tries to follow more strictly the research, the other one tries to explore a slightly different path to achieve the same goal.

All methods will be confronted with both performance and visual appeal, at last conclusions will be drawn to determine whether the proposed methods are valid or not.

II. PROBLEM STATEMENT

Image inpainting is a fundamental computer vision task that aims to restore missing or damaged regions in images with visually plausible content.

Modern Deep learning techniques provide already results that are visual appealing, however there is still work to be done, mostly about computational efficiency.

The core idea behind this project is to develop a version of [PEPSI](#) consistent with the one described in [\[1\]](#) as well as to

implement an alternative method with the goal to increase efficiency, trying to maintain a certain grade of resulting appealing in generated images.

III. STATE OF THE ART

The current state of the art on image inpainting relies on [GAN](#)-based methods, more specifically the coarse-to-fine network with [CAM](#).

Coarse-to-fine is based on two stacked generative networks, the first block aims to provide a first coarse image, trained by using reconstruction loss. Such patch is then passed to the refinement block which uses CAM to enhance the coarse patch and increase its quality.

However, the computational cost for such quality is still high. PEPSI, as well as Diet-PEPSI, tries to explore different solutions to decrease the number of parameters used and so the cost by using only one encoding network and two parallel decoding networks.

Current GAN methods also use two different discriminators, global and local, the global one examines the whole image and looks for global inconsistencies, the local one on the other hand examines only the hole region and a small region around it, however the local one only works on rectangular holes and this makes it difficult to apply in real world scenarios.

We can also distinguish between two different ideas on inpainting techniques, the traditional ones and the deep learning-based ones.

Traditional ones include diffusion-based and patch-based, the first tries to fill holes by propagating the image appearance around the hole, patch-based tries to sample different regions of the non-hole region and then combines them to fill the hole.

Deep learning-based ones, by using [CNNs](#), try to learn useful parameters to recognize and then reconstruct structures in the hole regions.

IV. PROPOSED METHOD

The proposed method is based on GAN, consists of a Generator (G) and a Discriminator (D) competing against each other, G is trained to produce a new image, indistinguishable from the real ones while D is optimized to differentiate between real and fake images, they do so by respectively minimizing and maximizing the loss function:

$$\min_G \max_D E_{x \sim P_{data}(x)} [\log D(x)] + E_{z \sim P_{z(z)}} [\log(1 - D(G(z)))]$$

Where z and x denote a random noise vector and a real image sampled from the noise $P_{z(z)}$ and real data distribution $P_{data}(x)$, respectively.

In the project, two different approaches have been taken, I decided to address as “PEPSIPP” the one more loyal to the referenced paper and “PEPSI” the one

with slightly different development approach.

In PEPSIPP, as suggested in the Diet-PEPSI model in the paper, a rate-adaptive dilated convolutional layer will be used, so that both the coarse and refinement path will share the same parameters. As suggested in [\[1\]](#) the weights of the convolutional layer are four-dimensional tensors:

$$W \in \mathbb{R}^{k \times k \times C_{in} \times C_{out}}$$

where k is the kernel-size, while C_{in} and C_{out} are the number of input and output channels respectively, which means that to produce different features we modulate W using a scale factor

$$\gamma_d \in \mathbb{R}^{1 \times 1 \times C_{in} \times C_{out}}$$

and a bias

$$\beta_d \in \mathbb{R}^{1 \times 1 \times C_{in} \times C_{out}}$$

resulting with

$$W_d = \gamma_d * W + \beta_d$$

This is supposed to largely reduce the number of parameters used and speed up the process.

Basically the image with the hole is first passed through the coarse process which produces a coarse version of the patched image, then the resulting image is passed through the refinement process to produce a more valid output image, both processes share the same adaptive block for which I decided to keep a base scale factor of “1” for all the parameters.

In the training process I use both the coarse and the refined images to evaluate the loss, the loss formula applied for this case is the same one suggested in [\[1\]](#):

$$L_G = \frac{\lambda_i}{N} \sum_{n=1}^N \|X_i^{(n)} - Y^{(n)}\| - \lambda_{adv} E_{x \sim P_{X_i}} [D(x)]$$

Where $X_i^{(n)}$ and $Y^{(n)}$ represent the n-th image pair of the generated image through the inpainting path and its corresponding original image and λ_i and λ_{adv} are hyperparameters which control the relative importance of each loss term, to this term we add the specific loss for the coarse path, defined as follows:

$$L_c = \frac{1}{N} \sum_{n=1}^N \|X_c^{(n)} - Y^{(n)}\|$$

where $X_c^{(n)}$ are the n-th image pair of the generated image via the coarse path.

The resulting loss is a combination of the previous two where the contribution given from the coarse path is going to be less considered as the training process goes on:

$$L_{tot} = L_G + \lambda_c (1 - \frac{k}{k_{maz}}) L_c.$$

On the other hand, for the new personal implementation “PEPSI” I decided to stay with a single generative procedure, in which I tried to differentiate the local and global features learning by creating two different branches, each on with its own loss, so that the two would specialize in recognizing only some specific features, then those features would be fused together in order to generate the final inpainted image in this case for the loss I tried to fit in various contributions:

•) The first one, called “recon loss” is just a L1 loss as defined before with the idea of reducing the general difference between fake and generated images:

$$L_{rec} = \sum_{n=1}^N \|X_i^{(n)} - Y^{(n)}\|$$

•) A second one, called “perceptual loss”, more complex than the first, has been evaluated by using vgg16, a very useful model used for shape and pattern recognition, I explicitly used only the pretrained one and set it so that it would not update during the process, so that I could be sure that the vgg model would not update itself, compromising the training process of the proposed model, this loss would result from the contribution of the vgg16 loss evaluated only in some specific layers.

•) The third contribution comes from evaluating the results from the status of the discriminator, the idea is basically that the generator will try to maximize the times the discriminator fails to recognize a fake image.

•) The fourth contribution comes from the structure branch. The idea is that the features extracted from the structure branch, since they are supposed to describe only the general context, get blurred, to remove any relation to specific textures then a simple L1 loss is evaluated between such results and the original image.

•) The fifth and final contribution comes from the texture branch, in this case we evaluate the loss using vgg16 once again since given its reliability on evaluating borders, corners and shapes in general.

All these contributions are then summed all up, each with its own scale factor.

Lastly, for the discriminator we used the base formula:

$$L_D = E_{x \sim P_Y} [\max(0, 1 - D(x))] + E_{x \sim P_{X_i}} [\max(0, 1 + D(x))]$$

where P_{X_i} and P_Y denote the data distributions of inpainting results and input image.

Regardless of the model, both G and D update their weights and parameters using Adam optimizers and scalers which ensure that the gradients will be correctly updated avoiding some problems which could derive from the use of mixed precision.

It's worth noting that I've also tried to implement two basic versions inspired by partial convolution and lama, mostly to give a basic idea of the capabilities of such methods and confront them with the two main methods, for both lama and partial convolution the loss is the result of the same contributions just described, excluding the last two.

As last thing in the training process, every some epochs (defined by a configurable variable) the network status is saved by saving a checkpoint containing the state for both the Generator and the Discriminator as well as the state of the two Adam optimizers, also the first 4 images of the first 8 batches will be saved in their versions [masked, inpainted, original] to have a first basic visual evaluation metric, which will be better exploited later on the evaluation step.

V. DATASET AND SETUP

Since using both COCO and PLACES2 datasets in their totality would have required too much computational power, a subset has been taken for both of them, for the training process, in order to maintain a certain grade of differentiation in the images a subset of 9555 images have been taken from COCO, on the other hand for the evaluation was not necessary to have a dataset as big, so a subset of just 496 images has been taken from PLACES2.

The experiments have all been conducted with the random seed set at 42 to ensure that randomness would not interfere differently with the proposed model, partially invalidating the results.

Since the experiment has been conducted by using an RTX 4070 SUPER gpu, after some tests I decided to train the various models for 150 epochs with batch sizes of 16 and no parallel workers, saving the training status every 50 epochs.

For the scaling factors tuning, two different choices have been made: for those whose values were suggested in [\[1\]](#) and for those who had a clear description of the consequences of setting them to specific values this rules have been followed (like for example the learning rates of both D and G and the betas for the Adam optimizers); for those who this data where not available I decided to set some arbitrary values (like for example with scale factors of the coarse and refinement paths)

VI. MODEL EVALUATION

```
C:\Users\ico19\Desktop\PepsiPP>py -m evaluation --ckpt C:\Users\ico19\Desktop\PepsiPP\checkpoints\ckpt_partial_conv_epoc
h150.pth --model partial_conv --dataset C:\Users\ico19\Desktop\PepsiPP\data\val_data
PSNR: 21.0647665928586 SSIM: 0.8278934206218983
Saved generated images in eval_out_partial_conv
To compute FID: pytorch-fid <path real images> <path generated>
```

Fig. 1 output of partial convolution evaluation process, showing its PSNR and SSIM values

```
C:\Users\ico19\Desktop\PepsiPP>py -m evaluation --ckpt .\checkpoints\ckpt_pepsi_pp_epoch150.pth --model pepsi_pp --datas
et .\data\val_data
PSNR: 28.708076333832402 SSIM: 0.9178921830893677
Saved generated images in eval_out_pepsi_pp
To compute FID: pytorch-fid <path real images> <path generated>
```

Fig. 2 output of *pepsi++* evaluation process, showing its PSNR and SSIM values

```
C:\Users\ico19\Desktop\PepsiPP>py -m evaluation --ckpt .\checkpoints\ckpt_pepsi_epoch150.pth --model pepsi --dataset .\data\val_data
PSNR: 27.59225561517215 SSIM: 0.9141775682367578
Saved generated images in eval_out_pepsi
To compute FID: pytorch-fid <path real images> <path generated>
```

Fig. 3 output of pepsi evaluation process, showing its PSNR and SSIM values

```
C:\Users\ico19\Desktop\PepsiPP>py -m evaluation --ckpt .\checkpoints\ckpt_lama_like_epoch150.pth --model lama_like --dataset .\data\val_data
PSNR: 25.863352747339743 SSIM: 0.8962454379559907
Saved generated images in eval_out_lama_like
To compute FID: pytorch-fid <path real images> <path generated>
```

Fig. 4 output of lama like evaluation process, showing its PSNR and SSIM values

From the evaluation method it is clear to see how the two PEPSI implementations result in a better value for both SSIM and PSNR, more precisely, compared to those resulting from [1], it is safe to say that the proposed implementations produce a

valid result. Looking at the FID value is once again clear the expected validity of the two proposed methods, in particular, the one that mostly resembles the guidelines in [1] has a much lower FID value, as shown in Fig.6:

```
C:\Users\ico19\Desktop\PepsiPP>pytorch-fid .\eval_out_partial_conv\orig .\eval_out_partial_conv\generated
100%|██████████████████████████████████████████████████████████████████████████████| 10/10 [00:05<00:00, 1.69it/s]
100%|██████████████████████████████████████████████████████████████████████████████| 10/10 [00:06<00:00, 1.57it/s]
FID: 71.47661416601136
```

Fig 5. FID evaluation for partial convolution

```
C:\Users\ico19\Desktop\PepsiPP>pytorch-fid .\eval_out_pepsi_pp\orig .\eval_out_pepsi_pp\generated
100%|██████████████████████████████████████████████████████████████████████████| 10/10 [00:06<00:00, 1.62it/s]
100%|██████████████████████████████████████████████████████████████████████████| 10/10 [00:06<00:00, 1.64it/s]
FID: 28.61914999283522
```

Fig 6. FID evaluation for pepsi++


```
C:\Users\ico19\Desktop\PepsiPP>pytorch-fid .\eval_out_pepsi\orig .\eval_out_pepsi\generated
100%|████████████████████████████████████████████████████████████████████████████████| 10/10 [00:06<00:00, 1.61it/s]
100%|████████████████████████████████████████████████████████████████████████████████| 10/10 [00:06<00:00, 1.63it/s]
FID: 35.32110912609005
```

Fig 7. FID evaluation for pepsi

```
C:\Users\ico19\Desktop\PepsiPP>pytorch-fid .\eval_out_lama_like\orig .\eval_out_lama_like\generated
100%|████████████████████████████████████████████████████████████████████████████████| 10/10 [00:07<00:00, 1.38it/s]
100%|████████████████████████████████████████████████████████████████████████████████| 10/10 [00:07<00:00, 1.38it/s]
FID: 35.51312826933554
```

Fig 8. FID evaluation for lama like

Comparing now the visual appearance of the various methods with box square and irregular holes we can see how, once

again, the two proposed models produce a pretty good result given the nonprofessional instruments:



Fig 9. Comparing in order: the original image, the square masked image, the inpainted image using pepsi++ then pepsi, then with partial convolution and lastly with lama like



Fig 10. Comparing in order: the original image, the free form masked image, the inpainted image using pepsi++ then pepsi, then with partial convolution and lastly with lama like

Is clear that the PEPSI++ method is the one working best on square holes, PEPSI is also trying to produce some results while both partial convolution and lama result in an unacceptable result.

With free form masks we have slightly different results, PEPSI++ is still the best method but both PEPSI and lama produce some kind of acceptable results.

Lastly, this is worth noting that while PEPSI++ gives the best results, it also is the one with the slowest training process, this is obviously given by the fact that its

implementation is much more complex than the other models.

VII. CONCLUSIONS

As said above, PEPSI++ gives the best results, at cost of having also the slowest training process, around 20% slower than the proposed PEPSI method.

It is also worth mentioning that all the training process has been done with

default values, given the proper amount of time on fine tuning the configuration parameters would result in much better performances, once again proving the validity of PEPSI++ while making PEPSI a valid option worth on further exploration.

VIII. REFERENCES

- [1] Li, W., Lin, W., Chen, L., Li, J., & Tang, C. (2022). PEPSI++: Fast and Lightweight Network for Image Inpainting. arXiv preprint arXiv: 1905.09010
- [2] PEPSI: Parallel Extended-decoder Path for Semantic Inpainting network
The proposed structure to implement
- [3] GAN: Generative Adversarial Network
Based on Generator and Discriminator
competing against each other
- [4] CAM: Contextual Attention Module, better explained in “Yu, J., Lin, Z., Yang, J., Shen, X., Lu, X., & Huang, T. S. (2018). Generative image inpainting with contextual attention. In Proceedings of the IEEE conference on computer vision and pattern recognition”
- [5] CNN: Convolutional Neural Network, a type of feedforward neural network that learns features via filter optimization